

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE Nº 08

NOMBRE COMPLETO: PABLO SEBASTIAN NOYOLA TORRES

Nº de Cuenta: 320023213

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 07

SEMESTRE 2027 - 1

FECHA DE ENTREGA LÍMITE: 13 de octubre de 2026

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

1. Actividades realizadas. Una descripción de los ejercicios y capturas de pantalla de bloques de código generados y de ejecución del programa

1.- Hacer que al presionar una tecla, la luz del faro del avión que apunta hacia el suelo cicle de color entre rojo, amarillo, verde, cian, azul, magenta.

Capturas de código:

P08-320023213.cpp

Cambio de color automático (Por tiempo), con solo tocar una vez una tecla:

```
976         // Avion--------------------------------------------------------
977
978         // Faro 1
979         // Cambio de color automatico por tiempo
980         static float contadorTiempo = 0.0f;
981         float tiempoCambio = 45.0f;
982
983         if (mainWindow.getAutoColorAvion())
984         {
985             contadorTiempo += deltaTime;
986             if (contadorTiempo >= tiempoCambio)
987             {
988                 int indiceActual = mainWindow.getColorIndiceAvion();
989                 indiceActual = (indiceActual + 1) % 6;
990                 mainWindow.setColorIndiceAvion(indiceActual);
991                 contadorTiempo = 0.0f;
992             }
993         }
994         else
995         {
996             contadorTiempo = 0.0f;
997         }
```

Cambio de color por pulsación de tecla:

```
998
999         // Cambio de color por tecla
1000        int indiceColor = mainWindow.getColorIndiceAvion();
1001        glm::vec3 colorSeleccionado = coloresCiclo[indiceColor];
```

```
1003        model = glm::mat4(1.0f);
1004        model = glm::translate(model, glm::vec3(-40.0f, 5.0f + mainWindow.getmueveyavion(), 6.0f + mainWindow.getmuevexavion()));
1005        model = glm::scale(model, glm::vec3(8.0f, 8.0f, 8.0f));
1006        glm::vec4 posicionLocalFaroAvion = glm::vec4(0.0f, -0.06f, 0.0f, 1.0f);
1007        glm::vec3 posicionMundoFaroAvion = glm::vec3(model * posicionLocalFaroAvion);
1008        glm::vec4 direccionLocalFaroAvion = glm::vec4(0.0f, -1.0f, 0.0f, 0.0f);
1009        glm::vec3 direccionMundoFaroAvion = glm::vec3(model * direccionLocalFaroAvion);
1010        direccionMundoFaroAvion = glm::normalize(direccionMundoFaroAvion);
1011
1012        spotLights[3].SetFlash(posicionMundoFaroAvion, direccionMundoFaroAvion);
1013        spotLights[3].SetColor(colorSeleccionado);
```

SpotLight.h

```
23      void SetColor(glm::vec3 col); // Metodo añadido
```

SpotLight.cpp

```
54  // Metodo añadido
55  void SpotLight::SetColor(glm::vec3 col) {
56      color = col;
57  }
```

Window.h

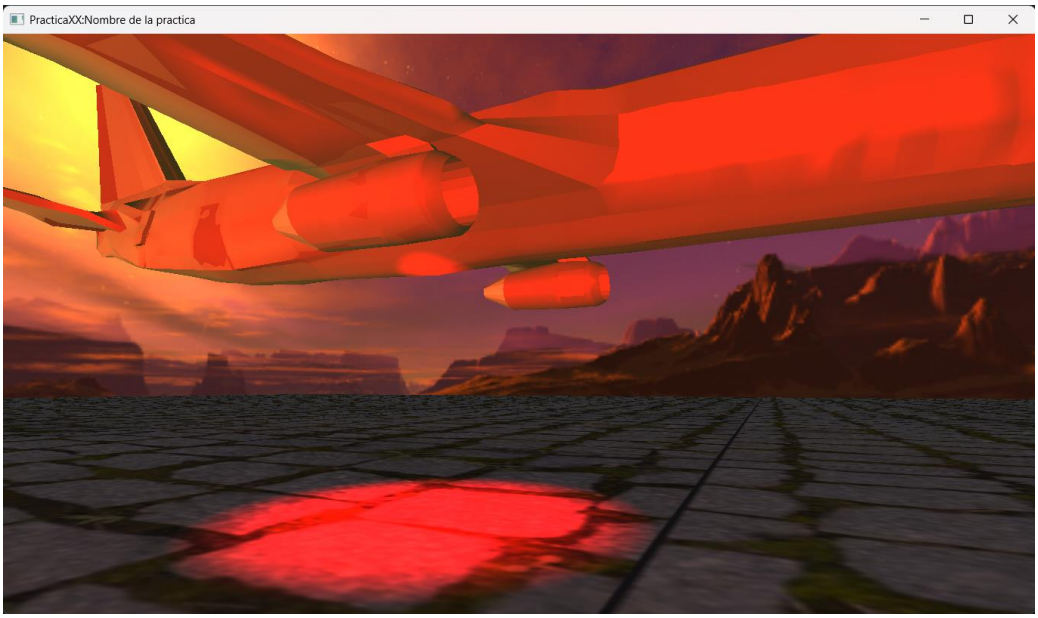
```
50      GLint getColorIndiceAvion() { return ColorIndiceAvion; }
51      bool getAutoColorAvion() { return AutoColorAvion; }
52      void setColorIndiceAvion(GLint indice) { ColorIndiceAvion = indice; }
53      bool getApagadorHolocron() { return ApagadorHolocron; }
54      bool getApagadorPinza() { return ApagadorPinza; }
55
56      ~Window();
57  private:
58      GLFWwindow *mainWindow;
59      GLint width, height, ColorIndiceAvion;
60      GLfloat articulacion1, articulacion2, articulacion3, articulacion4, articulacion5, articulacion6,
61          articulacion7, articulacion8, articulacion9, articulacion10, articulacion11, articulacion12, articulacion13,
62          articulacion14, articulacion15, articulacion16, articulacion17, articulacion18, articulacion19, articulacion20,
63          articulacion21, articulacion22, articulacion23, articulacion24, articulacion25, articulacion26, articulacion27;
64      bool keys[1024], AutoColorAvion, ApagadorHolocron, ApagadorPinza;
65      GLint bufferWidth, bufferHeight;
```

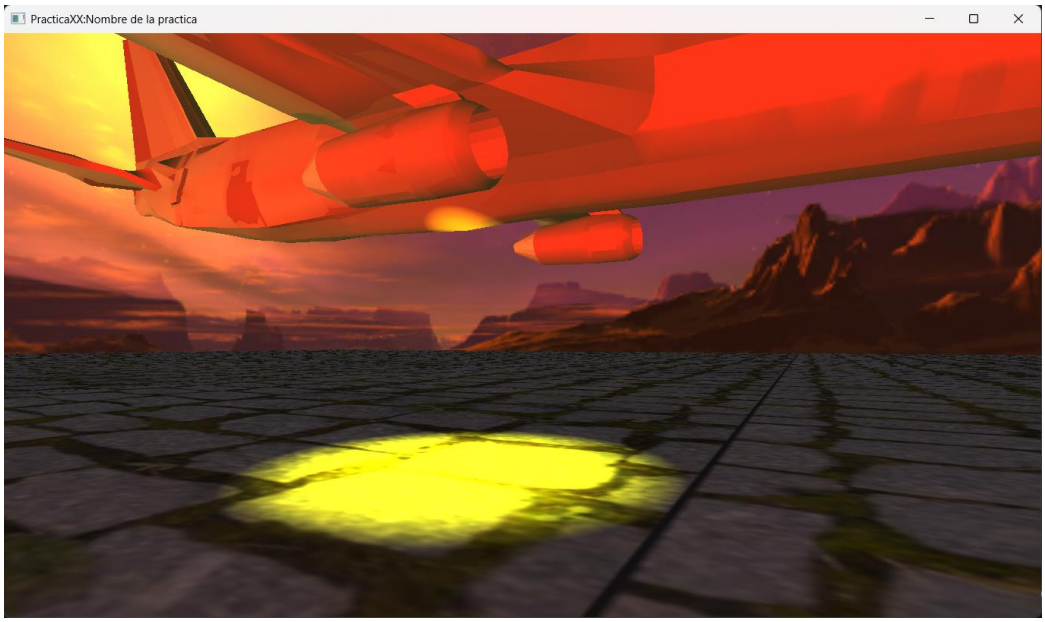
Window.cpp

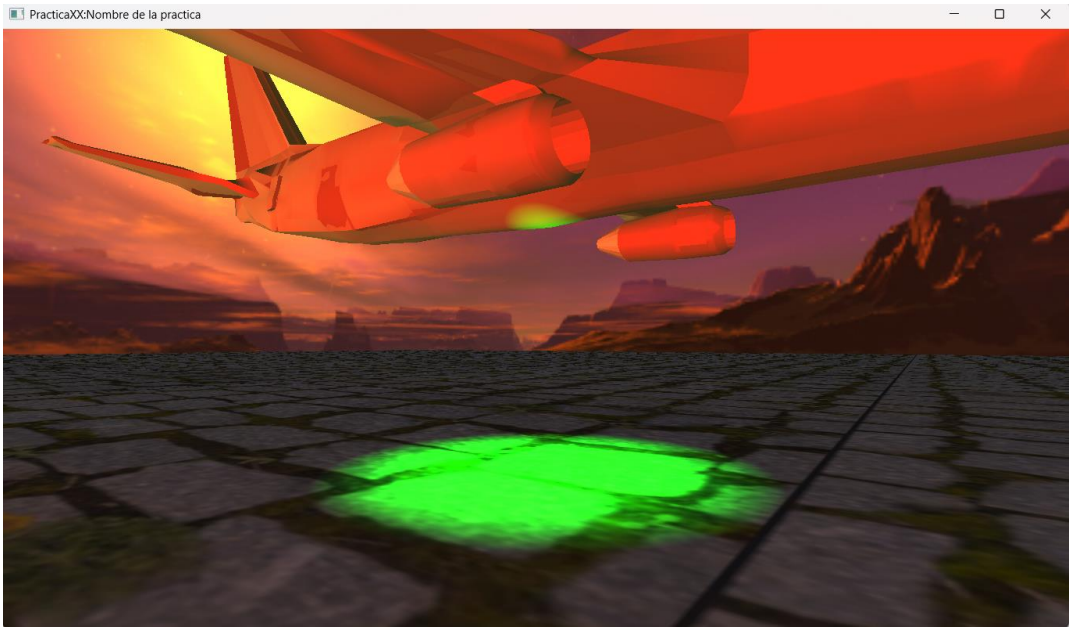
```
12  Window::Window(GLint windowWidth, GLint windowHeight)
13  {
14      width = windowWidth;
15      height = windowHeight;
16      ColorIndiceAvion = 0;
17      muevex = 2.0f;
18      articulacion1 = 0.0f;
19      articulacion2 = 0.0f;
20      articulacion3 = 0.0f;
21      articulacion4 = 0.0f;
22      articulacion5 = 0.0f;
23      articulacion6 = 0.0f;
24      articulacion7 = 0.0f;
25      articulacion8 = 0.0f;
26      articulacion9 = 0.0f;
27      articulacion10 = 0.0f;
28      articulacion11 = 0.0f;
29      articulacion12 = 0.0f;
30      articulacion13 = 0.0f;
31      articulacion14 = 0.0f;
32      articulacion15 = 0.0f;
33      articulacion16 = 0.0f;
34      articulacion17 = 0.0f;
35      articulacion18 = 0.0f;
36      articulacion19 = 0.0f;
37      articulacion20 = 0.0f;
38      articulacion21 = 0.0f;
39      articulacion22 = 0.0f;
40      articulacion23 = 0.0f;
41      articulacion24 = 0.0f;
42      articulacion25 = 0.0f;
43      articulacion26 = 0.0f;
44      articulacion27 = 0.0f;
45      mueveyavion = 2.0f;
46      muevexavion = 2.0f;
47      AutoColorAvion = false;
```

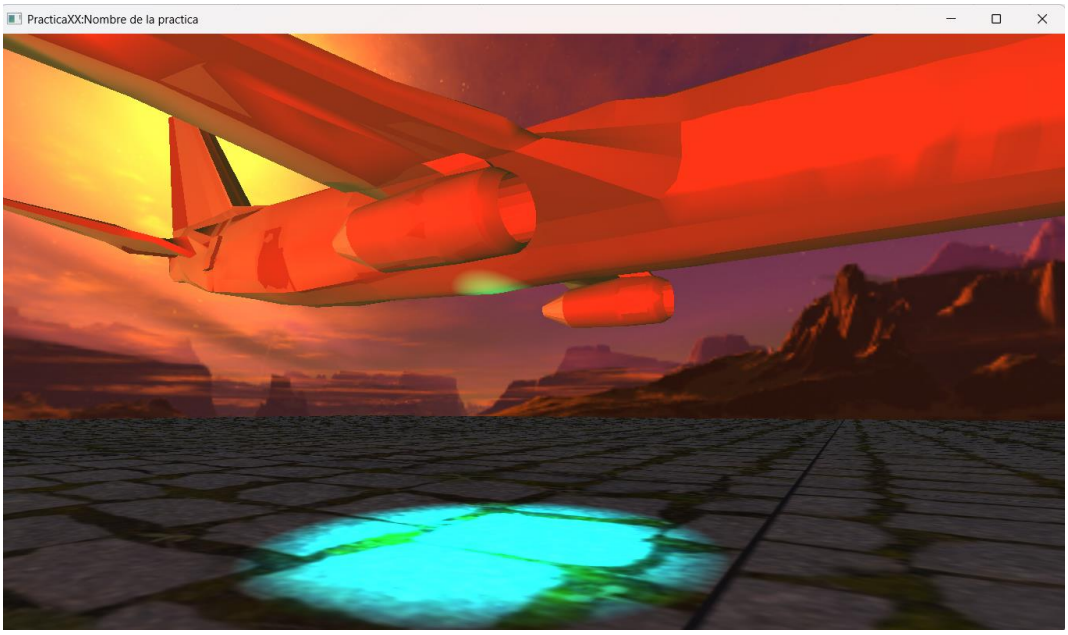
```
383     if (key == GLFW_KEY_T && action == GLFW_PRESS)
384     {
385         theWindow->ColorIndiceAvion = (theWindow->ColorIndiceAvion + 1) % 6;
386     }
387     if (key == GLFW_KEY_R && action == GLFW_PRESS)
388     {
389         theWindow->AutoColorAvion = !(theWindow->AutoColorAvion);
390     }
```

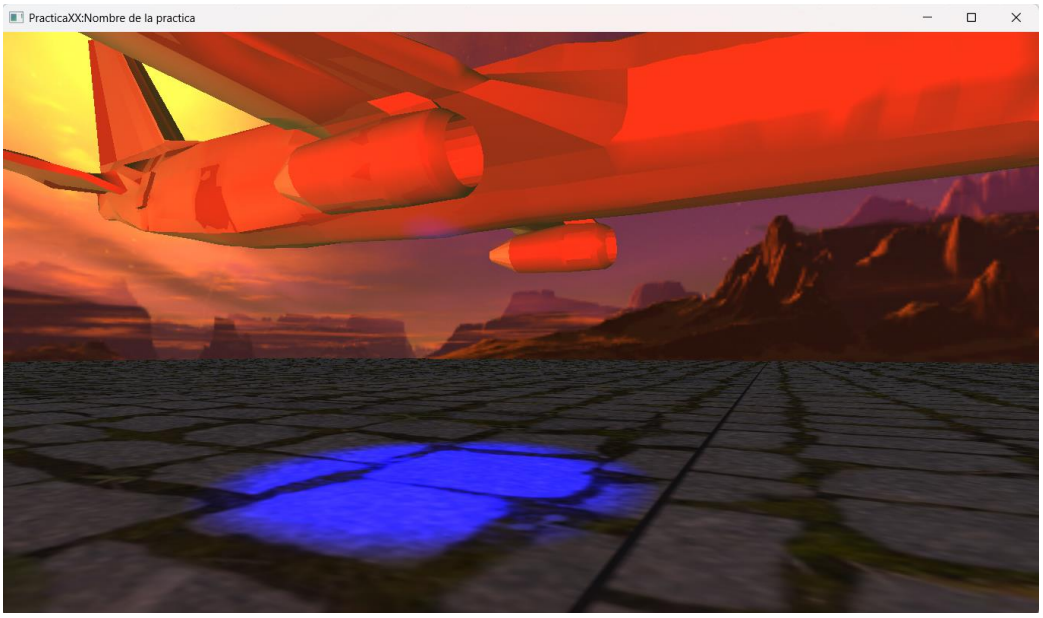
Capturas de ejecución:

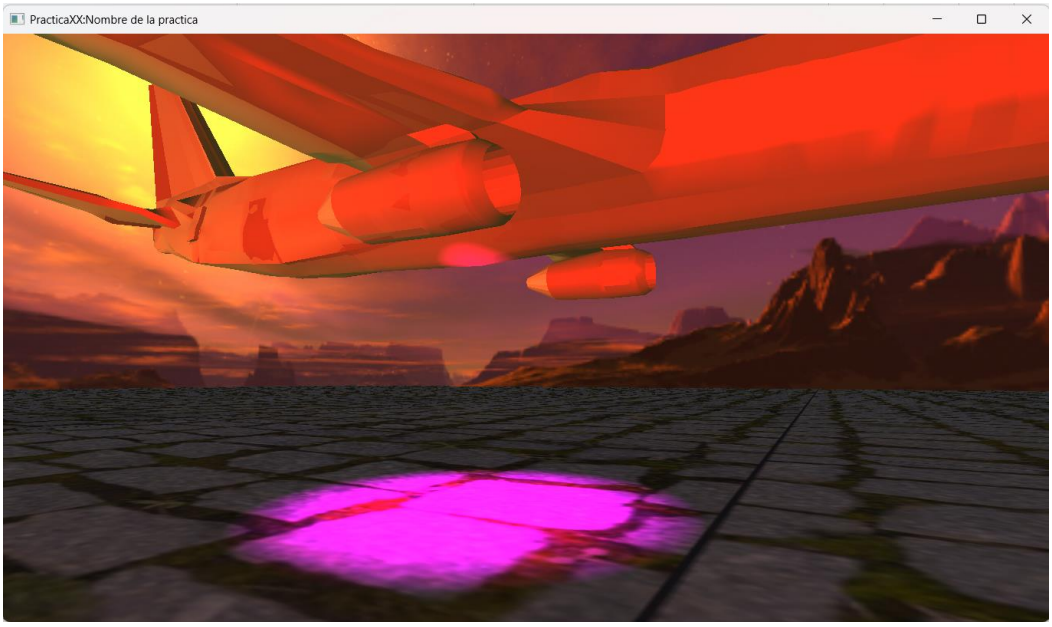





PracticaXX:Nombre de la practica


PracticaXX:Nombre de la practica





En este ejercicio se implementa una lógica doble de control de iluminación para el faro del avión, permitiendo alternar entre un cambio de color automático basado en un temporizador y un cambio manual mediante la pulsación de una tecla específica. Por un lado, mediante el uso de un contador de tiempo (contadorTiempo) que se acumula con deltaTime, se evalúa si el modo automático está activo (mainWindow.getAutoColorAvion()) para que al superar el valor establecido de 45.0f segundos, se actualice de forma cíclica el índice del color actual utilizando una operación de módulo seis ((indiceActual + 1) % 6) que recorre los tonos rojo, amarillo, verde, cian, azul y magenta. Por otro lado, la interacción manual se gestiona directamente en los callbacks de teclado de la clase Window, donde al presionar la tecla T se incrementa el índice de color de manera manual y al presionar

la tecla R se cambia el estado booleano que enciende o apaga el ciclo automático. Asimismo, se agregó el método SetColor (declarado en SpotLight.h y definido como void SpotLight::SetColor(glm::vec3 col) { color = col; } en SpotLight.cpp) con el fin de asignar a la clase de luz tipo reflector de una vía directa para modificar y actualizar su atributo de color interno en tiempo de ejecución. Esto se hizo porque la clase original no dejaba modificar el color en cada instante después de iniciarla, haciendo posible que al usar spotLights[3].SetColor(colorSeleccionado) la luz cambie al instante de tono, ya sea por el tiempo o cuando le piques a la tecla.

2.- Añadir a la posición del Holocron importado una luz puntual de color azul claro.

4.- Presionando una tecla prender y apagar la luz unida al Holocron.

Capturas de código:

P08-320023213.cpp

```
942         // Holocron importado de blender
943         bool holocronEncendido = mainWindow.getApagadorHolocron();
944         float intensidadAmbienteH = holocronEncendido ? 0.8f : 0.0f;
945         float intensidadDifusaH = holocronEncendido ? 1.2f : 0.0f;
946         glm::vec3 colorLuzH = holocronEncendido ? glm::vec3(0.3f, 0.9f, 1.0f) : glm::vec3(0.0f, 0.0f, 0.0f);
947         pointLights[1] = PointLight(
948             colorLuzH.r, colorLuzH.g, colorLuzH.b,
949             intensidadAmbienteH,
950             intensidadDifusaH,
951             14.5f, 2.5f, 25.0f,
952             1.0f,
953             0.09f,
954             0.032f
955         );
956         glm::vec3 colorHolocron = holocronEncendido ? glm::vec3(0.3f, 0.9f, 1.0f) : glm::vec3(1.0f, 1.0f, 1.0f);
957         color = colorHolocron;
958         glUniform3fv(uniformColor, 1, glm::value_ptr(color));
959         model = glm::mat4(1.0f);
960         model = glm::translate(model, glm::vec3(14.5f, 2.5f, 25.0f));
961         model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
962         glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
963         Holocron_M.RenderModel();
```

Windows.h

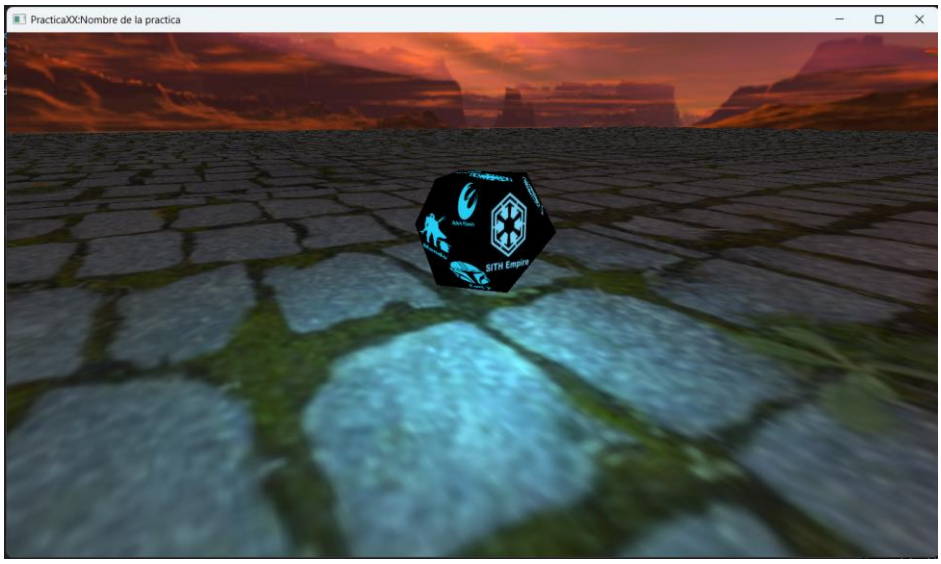
```
53      bool getApagadorHolocron() { return ApagadorHolocron; }
54      bool getApagadorPinza() { return ApagadorPinza; }
55
56      ~Window();
57  private:
58      GLFWwindow *mainWindow;
59      GLint width, height, ColorIndiceAvion;
60      GLfloat articulacion1, articulacion2, articulacion3, articulacion4, articulacion5, articulacion6,
61          articulacion7, articulacion8, articulacion9, articulacion10, articulacion11, articulacion12, articulacion13,
62          articulacion14, articulacion15, articulacion16, articulacion17, articulacion18, articulacion19, articulacion20,
63          articulacion21, articulacion22, articulacion23, articulacion24, articulacion25, articulacion26, articulacion27;
64      bool keys[1024], AutoColorAvion, ApagadorHolocron, ApagadorPinza;
```

Window.cpp

```cpp
Window::Window(GLint windowWidth, GLint windowHeight)
{
    width = windowWidth;
    height = windowHeight;
    ColorIndiceAvion = 0;
    muevex = 2.0f;
    articulacion1 = 0.0f;
    articulacion2 = 0.0f;
    articulacion3 = 0.0f;
    articulacion4 = 0.0f;
    articulacion5 = 0.0f;
    articulacion6 = 0.0f;
    articulacion7 = 0.0f;
    articulacion8 = 0.0f;
    articulacion9 = 0.0f;
    articulacion10 = 0.0f;
    articulacion11 = 0.0f;
    articulacion12 = 0.0f;
    articulacion13 = 0.0f;
    articulacion14 = 0.0f;
    articulacion15 = 0.0f;
    articulacion16 = 0.0f;
    articulacion17 = 0.0f;
    articulacion18 = 0.0f;
    articulacion19 = 0.0f;
    articulacion20 = 0.0f;
    articulacion21 = 0.0f;
    articulacion22 = 0.0f;
    articulacion23 = 0.0f;
    articulacion24 = 0.0f;
    articulacion25 = 0.0f;
    articulacion26 = 0.0f;
    articulacion27 = 0.0f;
    mueveyavion = 2.0f;
    muevexavion = 2.0f;
    AutoColorAvion = false;
    ApagadorHolocron = true;
    ApagadorPinza = true;
```

```cpp
    if (key == GLFW_KEY_E && action == GLFW_PRESS)
    {
        theWindow->ApagadorHolocron = !(theWindow->ApagadorHolocron);
    }
```

Capturas de ejecución:





Para el segundo ejercicio, se añadió una luz puntual de color azul claro configurando un objeto PointLight en el arreglo pointLights[1], posicionándolo exactamente en las coordenadas (14.5f, 2.5f, 25.0f) que corresponden a la ubicación del holocrón importado, y asignándole un tono RGB glm::vec3(0.3f, 0.9f, 1.0f) junto con sus respectivas intensidades de iluminación ambiental y difusa. Para el cuarto ejercicio, se implementó el mecanismo para prender y apagar dicha luz mediante una tecla integrando la variable booleana ApagadorHolocron en la clase Window (la cual inicia en true), la cual se invierte al presionar la tecla E por medio del evento GLFW_PRESS; de esta manera, en el bucle principal del programa se evalúa este estado para que, al estar activo, la luz proyecte su color característico, y al desactivarse, las intensidades se reduzcan a cero para apagar por completo la luz unida al modelo del holocrón. También es importante mencionar que la línea glm::vec3 colorHolocron = holocronEncendido ? glm::vec3(0.3f, 0.9f, 1.0f) : glm::vec3(1.0f, 1.0f, 1.0f); se agregó para cambiar directamente el tinte o color propio del objeto 3D del holocrón en el shader mediante el uniforme de color; esto se hace para que el modelo mismo cambie visualmente a un tono azul resplandeciente cuando la luz está prendida y regrese a su color normal original de la textura cuando se apague, mejorando la sensación visual de que la pieza está encendida o apagada.

3.- Añadir en el punto extremo de la pinza de rover una luz puntual de color verde.

5.- Presionando una tecla prender y apagar la luz unida a la pinza del Rover.

Capturas de código:

P08-320023213.cpp

```cpp
// Parte brazo 4
bool pinzaEncendida = mainWindow.getApagadorPinza();
glm::vec3 colorPinza = pinzaEncendida ? glm::vec3(0.0f, 1.0f, 0.3f) : glm::vec3(0.5f, 0.5f, 0.5f);
color = colorPinza;
glUniform3fv(uniformColor, 1, glm::value_ptr(color));

glm::mat4 modelParteBrazo4 = modelParteBrazo3;
modelParteBrazo4 = glm::translate(modelParteBrazo4, glm::vec3(0.0f, 0.1f, 0.02f));
modelParteBrazo4 = glm::rotate(modelParteBrazo4, glm::radians(mainWindow.getarticulacion5()), glm::vec3(0.0f, 0.0f, 1.0f));

glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(modelParteBrazo4));
Rover_partebrazo4.RenderModel();

// Para la posición de la pinza
glm::vec4 posicionLocalPinza = glm::vec4(0.0f, 0.2f, 0.0f, 1.0f);
glm::vec3 posicionMundoPinza = glm::vec3(modelParteBrazo4 * posicionLocalPinza);

float intensidadAmbienteP = pinzaEncendida ? 0.2f : 0.0f;
float intensidadDifusaP = pinzaEncendida ? 1.5f : 0.0f;
glm::vec3 colorLuzP = pinzaEncendida ? glm::vec3(0.0f, 1.0f, 0.2f) : glm::vec3(0.0f, 0.0f, 0.0f);

pointLights[2] = PointLight(
    colorLuzP.r, colorLuzP.g, colorLuzP.b,
    intensidadAmbienteP,
    intensidadDifusaP,
    posicionMundoPinza.x, posicionMundoPinza.y, posicionMundoPinza.z,
    1.0f,
    0.09f,
    0.032f
);
pointLightCount++;
color = glm::vec3(1.0f, 1.0f, 1.0f);
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
// ----------
```

Window.h

```cpp
bool getApagadorPinza() { return ApagadorPinza; }

~Window();
private:
GLFWwindow *mainWindow;
GLint width, height, ColorIndiceAvion;
GLfloat articulacion1, articulacion2, articulacion3, articulacion4
    articulacion7, articulacion8, articulacion9, articulacion10, a
    articulacion14, articulacion15, articulacion16, articulacion17
    articulacion21, articulacion22, articulacion23, articulacion24
bool keys[1024], AutoColorAvion, ApagadorHolocron, ApagadorPinza;
```

Window.cpp

```cpp
ApagadorPinza = true;
```

```cpp
if (key == GLFW_KEY_SPACE && action == GLFW_PRESS)
{
    theWindow->ApagadorPinza = !(theWindow->ApagadorPinza);
}
```

Capturas de pantalla:





Para el tercer ejercicio, se añadió una luz puntual de color verde en el punto extremo de la pinza del rover, calculando su posición en el mundo al multiplicar una coordenada local glm::vec4(0.0f, 0.2f, 0.0f, 1.0f) por la matriz de transformación del cuarto brazo (modelParteBrazo4), y configurando el objeto PointLight en pointLights[2] con un tono verde característico (glm::vec3(0.0f, 1.0f, 0.2f)) junto con sus respectivas atenuaciones e intensidades ambiental y difusa. Para el quinto ejercicio, se implementó el mecanismo para prender y apagar dicha luz mediante una tecla integrando la variable booleana ApagadorPinza en la clase Window (la cual inicia en true) y que cambia al presionar la barra espaciadora (GLFW_KEY_SPACE) por medio del evento GLFW_PRESS, de esta forma, en el bucle principal se evalúa este estado para que, al estar activo, la luz proyecte su color verde, y al desactivarse, las intensidades se reduzcan a cero para apagar por completo la iluminación de la pinza. También es importante mencionar que la línea

glm::vec3 colorPinza = pinzaEncendida ? glm::vec3(0.0f, 1.0f, 0.3f) : glm::vec3(0.5f, 0.5f, 0.5f); se agregó para modificar directamente el color propio del modelo 3D de la pinza en el shader mediante el uniforme de color, esto se hace para que la geometría misma cambie visualmente a un tono verdoso cuando la luz está encendida y regrese a un tono gris cuando se apague, mejorando el efecto visual del estado de iluminación de la pinza del Rover.

Video de ejecución:

https://www.youtube.com/watch?v=9jFdj6FjGHk

2. Problemas presentados. Listar si surgieron problemas a la hora de ejecutar el código

No se presentaron mayores problemas mas que el factor del tiempo, pues llevo de un cierto análisis principalmente de conceptos de programación para realizar los ejercicios solicitados.

3. Conclusión:
    a. Los ejercicios de la clase: Complejidad, explicación
    b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias.

Tras la realización de los ejercicios solicitados de iluminación, considero que fueron prácticas bastante entretenidas e interesantes para reforzar los conceptos teóricos de este tema de forma aplicada. No obstante, considero que haber contado con un poco más de tiempo para realizarlos de manera personal habría permitido profundizar aún más en cada detalle del desarrollo y la experimentación con las luces. A pesar de ello, la actividad resultó muy enriquecedora para comprender el comportamiento de las diferentes fuentes de iluminación en OpenGL y su interacción dinámica con los objetos de la escena.

Referencias:

Khronos OpenGL Working Group. (s. f.). OpenGL - The Industry Standard for High Performance Graphics. The Khronos Group. Recuperado de https://www.opengl.org/

OpenGL Mathematics (GLM). (s. f.). GLM: C++ mathematics library for graphics software. GitHub. Recuperado de https://github.com/g-truc/glm

GLFW. (s. f.). GLFW: An OpenGL desktop development library. Recuperado de https://www.glfw.org/

GLEW: The OpenGL Extension Wrangler Library. (s. f.). The OpenGL Extension Wrangler Library. SourceForge. Recuperado de http://glew.sourceforge.net/

Assimp (Open Asset Import Library). (s. f.). Assimp: A portablibe Open Source library to import various well-known 3D model formats in an uniform manner. GitHub. Recuperado de https://www.assimp.org/